

Hyena cho tiếng Việt: Mô hình ngôn ngữ tích chập dài dưới bậc hai, phân tích ablation và khởi tạo bộ lọc từ cấu trúc thông tin của corpus

Nguyễn Cao Trung Kiên Tô Huỳnh Minh Tiến Trần Tú Quang

Trường Đại học Công nghệ Thông tin, ĐHQG-HCM
{250201069, 250201095, 250201084}@grad.uit.edu.vn

Tóm tắt nội dung

Toán tử Hyena (Poli et al., 2023) thay thế self-attention bằng đệ quy đan xen tích chập dài tham số hoá ngầm và cổng điều khiển bởi dữ liệu, đạt chi phí dưới bậc hai theo độ dài chuỗi. Toàn bộ thực nghiệm ngôn ngữ trong bài báo gốc thực hiện trên tiếng Anh. Chúng tôi cài đặt lại Hyena, kiểm chứng tính đúng đắn của cài đặt bằng bộ kiểm thử tự động, và áp dụng cho mô hình ngôn ngữ tiếng Việt. Ngoài phần tái hiện, chúng tôi đề xuất khởi tạo phân bố suy giảm của bộ lọc Hyena từ đường cong suy giảm thông tin tương hỗ đo trên chính corpus huấn luyện, thay cho cách đặt thủ công. Ở cùng ngân sách tham số, cùng 50 triệu token và cùng siêu tham số, Hyena đạt perplexity thấp hơn Transformer 19,5% trên tiếng Việt và 14,7% trên tiếng Anh. Về hiệu năng, Hyena vượt kernel attention tiết kiệm bộ nhớ 1,88 lần ở độ dài 8192 cho lượt tiến và lùi, tiệm cận mức 2 lần mà bài báo gốc công bố. Về đóng góp đề xuất, chúng tôi đo được tiếng Việt mang lượng thông tin tương hỗ cục bộ gấp 1,7 đến 2,3 lần tiếng Anh ở khoảng cách ngắn và giảm dần về khoảng 1,0 ở rìa vùng đo được, nhưng nhánh đối chứng cho thấy bộ lọc suy ra từ hai ngôn ngữ gần nhau: chênh lệch tương đối trung bình là 14,3% nếu lấy độ dài hiệu dụng tiếng Anh làm mẫu số. Thực nghiệm xác nhận dự đoán đó: khởi tạo từ corpus chỉ hơn một đường cơ sở cách đều theo thang log 0,285 PPL với khoảng tin cậy chồng lấn, trong khi cả hai cùng vượt đường cơ sở đặt tùy tiện khoảng 0,9 PPL. Kết luận trung thực là khoảng giá trị của tham số suy giảm mới là điều quan trọng, còn việc suy nó từ thống kê corpus không mang lại lợi ích đo được ở quy mô này.

1 Giới thiệu

Cơ chế self-attention (Vaswani et al., 2017) có chi phí tính toán và bộ nhớ tăng theo bình

phương độ dài chuỗi, đặt ra giới hạn cứng cho lượng ngữ cảnh mà mô hình có thể xử lý. Nhiều hướng nghiên cứu tìm cách phá vỡ rào cản này, trong đó dòng mô hình không gian trạng thái (Gu et al., 2022; Mehta et al., 2022; Fu et al., 2023) và tích chập dài nổi lên như những lựa chọn thay thế nghiêm túc.

Hyena (Poli et al., 2023) là một trong những kiến trúc đầu tiên không dùng attention dày đặc mà vẫn đạt chất lượng tương đương Transformer trên mô hình hoá ngôn ngữ ở quy mô lớn. Toán tử này đan xen tích chập dài tham số hoá ngầm với cổng nhân điều khiển bởi dữ liệu, cho chi phí $O(NL \log_2 L)$ thay vì $O(L^2)$. Dòng nghiên cứu này về sau dẫn tới các kiến trúc chọn lọc trạng thái như Mamba (Gu and Dao, 2023).

Toàn bộ thực nghiệm ngôn ngữ trong bài báo gốc thực hiện trên tiếng Anh, cụ thể là WikiText103 (Merity et al., 2017) và The Pile (Gao et al., 2020). Chưa có bằng chứng nào về hành vi của Hyena trên tiếng Việt, một ngôn ngữ có đặc điểm chữ viết khác hẳn: chữ Quốc ngữ tách rời theo âm tiết, nên một từ ghép nhiều âm tiết bị chẻ thành nhiều token khi token hoá theo khoảng trắng. Các mô hình ngôn ngữ tiếng Việt hiện có phần lớn dựa trên kiến trúc Transformer (Nguyen and Nguyen, 2020).

Bài báo cáo này đóng góp ba điểm.

Thứ nhất, chúng tôi cài đặt lại toán tử Hyena bám sát đặc tả gốc và *chứng minh* tính đúng đắn của cài đặt bằng bộ kiểm thử tự động, trong đó có phép kiểm tính nhân quả dựa trên gradient kèm một phép kiểm ngược cố ý cài lỗi. Thứ hai, chúng tôi đo đường cong suy giảm thông tin tương hỗ trên corpus tiếng Việt và tiếng Anh, dùng nó để khởi tạo phân bố suy giảm của bộ lọc Hyena thay cho việc đặt tham số này bằng trực giác. Thứ ba, và đây là phần chúng tôi cho là có giá trị nhất, nhánh đối

chúng tiếng Anh *bác bỏ* giả thuyết ban đầu của chính chúng tôi rằng bộ lọc thu được mang tính đặc thù tiếng Việt; chúng tôi báo cáo đầy đủ kết quả phủ định này thay vì lược bỏ nó.

2 Nền tảng: toán tử Hyena

2.1 Định nghĩa

Gọi $u \in \mathbb{R}^{L \times D}$ là chuỗi đầu vào. Hyena bậc N trước hết tính $N + 1$ phép chiếu tuyến tính của đầu vào, tiếp theo là một tích chập ngắn theo kênh, thu được $(x^1, \dots, x^N, v)$. Toán tử được định nghĩa bằng đệ quy (Định nghĩa 3.1 trong Poli et al., 2023):

$$\begin{aligned} z_t^1 &= v_t \\ z_t^{n+1} &= x_t^n \cdot (h^n * z_t^n), \quad n = 1 \dots N \\ y_t &= z_t^{N+1} \end{aligned} \quad (1)$$

trong đó $*$ là tích chập nhân quả dài và h^n là các bộ lọc học được. Mỗi tích chập được tính trong miền Fourier, cho độ phức tạp tổng cộng $O(NL \log_2 L)$.

Bài báo gốc chỉ ra rằng Hyena bậc 1 tương ứng với GSS (Mehta et al., 2022) và bậc 2 tương ứng với H3 (Fu et al., 2023), nên bậc N là một trục thiết kế có ý nghĩa chứ không phải tham số tùy tiện.

2.2 Bộ lọc tham số hoá ngẫu nhiên

Điểm mấu chốt khiến bộ lọc dài mà không tốn tham số theo L là cách tham số hoá ngẫu nhiên:

$$h_t = \text{Window}(t) \cdot (\text{FFN} \circ \text{PositionalEncoding})(t) \quad (2)$$

với $\text{Window}(t) = \exp\{-\alpha t\}$ và α thay đổi theo từng kênh độc lập, nhằm buộc các bộ lọc có độ dài khác nhau. Mạng FFN dùng kích hoạt tuần hoàn tần số cao (sine), theo dòng biểu diễn ngẫu nhiên (Sitzmann et al., 2020; Romero et al., 2022), để khắc phục thiên lệch tần số thấp của mạng nơ-ron. Tính nhân quả được bảo đảm bằng cách đánh giá bộ lọc tại $t = 0 \dots L - 1$ rồi đệm 0 cả tín hiệu lẫn bộ lọc tới độ dài ít nhất $2L - 1$ trước khi biến đổi Fourier.

Phân bố $\{\alpha_i\}$ được mô tả là “thay đổi theo kênh” nhưng bài báo gốc không quy định khoảng giá trị cụ thể, cũng không gắn nó với thống kê của ngôn ngữ đích. Đây chính là khe hở mà đóng góp của chúng tôi nhắm vào.

3 Khởi tạo bộ lọc từ cấu trúc thông tin của corpus

3.1 Động cơ

Với t chuẩn hoá về $[0, 1]$ trên L vị trí, kênh thứ i suy giảm còn $1/e$ tại token thứ khoảng L/α_i . Ta gọi đại lượng này là *độ dài hiệu dụng* của kênh. Câu hỏi đặt ra: nếu đặt tập độ dài hiệu dụng phủ đúng nơi thông tin dự đoán thực sự nằm trong corpus, mô hình có dùng ngân sách kênh hiệu quả hơn không?

3.2 Đo cái gì, và vì sao không dùng bộ tách từ

Phương án hiển nhiên là chạy bộ tách từ tiếng Việt rồi lấy phân bố số âm tiết mỗi từ. Chúng tôi bác bỏ phương án này vì nó phá hỏng nhánh đối chứng: tiếng Anh không có bộ tách từ tương đương, nên mọi khác biệt quan sát được đều có thể quy cho công cụ đo thay vì cho ngôn ngữ.

Thay vào đó chúng tôi đo trực tiếp thông tin tương hỗ giữa hai token cách nhau d vị trí:

$$I(d) = \sum_{x,y} p_d(x,y) \log \frac{p_d(x,y)}{p(x)p(y)} \quad (3)$$

Cùng một công thức, cùng một quy trình ước lượng, áp cho cả hai ngôn ngữ.

3.3 Xử lý độ chệch của ước lượng

Ước lượng hợp lý cực đại của thông tin tương hỗ luôn chệch dương trên mẫu hữu hạn, và độ chệch tăng theo bình phương cỡ bảng chữ cái; bỏ qua điều này sẽ dẫn tới kết luận rằng ngôn ngữ có corpus nhỏ hơn mang nhiều thông tin tiềm ẩn hơn. Chúng tôi khử độ chệch bằng cách trừ đường nền xáo trộn: xáo ngẫu nhiên corpus phá huỷ mọi cấu trúc thật, nên thông tin tương hỗ đo trên bản xáo chính là độ chệch. Ngoài ra chúng tôi thô hoá bảng chữ cái xuống K ký hiệu thường gặp nhất, làm giảm độ chệch theo K^2 ; thô hoá cho ước lượng thận trọng.

3.4 Hiệu chuẩn công cụ đo

Một phép đo chưa được hiệu chuẩn trên dữ liệu có đáp án biết trước thì không dùng để kết luận được. Chúng tôi kiểm công cụ đo trên bốn chuỗi nhân tạo. Trên chuỗi độc lập hoàn toàn, đáp án đúng là $I = 0$ và bộ đo còn dư 2,82% độ chệch. Trên chuỗi sao chép chu kỳ $k = 8$, nó đặt đỉnh đúng tại $d = 8$ và đỉnh đó nổi 84 lần trên

nền. Trên chuỗi Markov, nó cho $I(1) = 2,79$, giảm đơn điệu và tắt ở $d = 64$. Trên chuỗi suy biến toàn 0, nó báo lỗi và từ chối sinh α thay vì trả về một phân bố vô nghĩa.

3.5 Ánh xạ sang phân bố suy giảm

Chúng tôi coi $I(d)$ là khối lượng thông tin nằm tại khoảng cách d , chuẩn hoá thành một phân bố xác suất trên khoảng cách, rồi lấy D phân vị cách đều làm tập độ dài hiệu dụng $\{\ell_i\}$, cuối cùng đặt $\alpha_i = L/\ell_i$. Nơi tập trung nhiều thông tin sẽ được cấp nhiều kênh hơn.

Trong các lần chạy E4, file α của nhánh corpus được sinh từ phép đo BPE với $K = 500$, nhưng mô hình ngôn ngữ chính lại huấn luyện với đơn vị âm tiết, nên độ dài hiệu dụng đang được chuyển nguyên từ token BPE sang vị trí token âm tiết. Tỷ lệ ký tự trên mỗi token trong file đo là 3,851 so với 4,118 ở nhánh tiếng Việt và 4,155 so với 5,100 ở nhánh tiếng Anh, tức lệch thang khoảng 6,5% và 18,5%. Chúng tôi xem đây là hạn chế về diễn giải, không quy trực tiếp thành sai số perplexity vì chưa có thí nghiệm cô lập yếu tố này.

3.6 Hai đường cơ sở, và vì sao cần cả hai

Chúng tôi so sánh với hai đường cơ sở. Đường thứ nhất, ký hiệu **uniform**, đặt α cách đều tuyến tính trong một khoảng do chúng tôi chọn. Đường thứ hai, ký hiệu **logspace**, đặt độ dài hiệu dụng cách đều theo thang log từ 1 tới L .

Đường **logspace** tồn tại vì một lý do quan trọng. Phép đo cho thấy 80% tới 90% thông tin nằm trong khoảng cách 34 token, tùy ngôn ngữ và bộ token hoá, trong khi cấu hình **uniform** đặt *toàn bộ* 256 kênh ra xa hơn 64 token (Bảng 2). So sánh đề xuất với một cấu hình lệch xa như vậy là so với hình nộm: thắng cũng không chứng minh được gì. Mọi kết luận khoa học trong báo cáo này đều rút ra từ so sánh với **logspace**.

4 Thiết lập thí nghiệm

4.1 Nguyên tắc so sánh công bằng

Trong so sánh chính giữa Hyena và Transformer, mọi cấu hình chỉ khác nhau ở toán tử trộn token. Embedding, pre-LayerNorm, khối MLP, kết nối residual, weight tying, cách khởi tạo và lịch learning rate giữ nguyên vẹn như nhau. Nếu để nhiều thứ

khác nhau cùng lúc thì không thể quy chênh lệch perplexity cho toán tử. Riêng các nhánh ablation thay đổi cấu trúc bên trong toán tử nên không giữ nguyên tổng tham số: bậc $N = 1$ ít hơn cấu hình gốc 4,4%, bậc $N = 3$ nhiều hơn 4,4%, bỏ positional embedding ít hơn 1,7% (xem Bảng 4).

Chúng tôi cấp positional embedding học được cho cả hai loại mô hình: attention bắt buộc cần thông tin vị trí còn Hyena thì không, nên cấp cho một bên mà không cấp cho bên kia sẽ tạo ra hai biến thay đổi thay vì một; một nhánh ablation riêng kiểm tra Hyena có thật sự không cần. Huấn luyện tính theo ngân sách token cố định chứ không theo số epoch, vì so theo epoch sẽ thiên vị mô hình chạy nhanh hơn.

4.2 Mô hình và dữ liệu

Mô hình 4 lớp, $d_{\text{model}} = 256$, 8 đầu attention, $d_{\text{ff}} = 1024$, $L = 512$, từ điển 16k. Số tham số đo được: Hyena 7.553.280 và Transformer 7.383.552, lệch 2,30%.

Dữ liệu lấy từ Wikipedia tiếng Việt và tiếng Anh (bản 20231101). Nhánh tiếng Anh được lấy dư 110.000 tài liệu rồi cắt xuống đúng 38.250.964 token của nhánh tiếng Việt, nên hai nhánh có cùng số token huấn luyện duy nhất. Cả hai sau đó huấn luyện trên cùng 49.995.776 token, tức cùng 6.103 bước và cùng khoảng 1,31 epoch.

Chúng tôi khảo sát hai đơn vị token hoá. Thứ nhất là âm tiết, tức tách theo khoảng trắng, vốn là đơn vị tự nhiên của chữ Quốc ngữ. Thứ hai là BPE mức byte (Sennrich et al., 2016), huấn luyện trên chính corpus, không sinh token ngoài từ điển. Phân biệt này chỉ tồn tại ở dạng như vậy với tiếng Việt: với tiếng Anh, tách theo khoảng trắng cho ra đơn vị từ chứ không phải âm tiết, nên nhánh đối chứng phải được diễn giải tương ứng.

4.3 Bằng chứng cài đặt đúng

Một mô hình ngôn ngữ nhân quả cài sai, để rò rỉ thông tin từ tương lai, vẫn huấn luyện trơn tru và cho perplexity đẹp bất thường. Đây là lỗi im lặng nguy hiểm nhất của loại nghiên cứu này.

Chúng tôi khử nó bằng 67 phép kiểm tự động, trong đó phép kiểm cốt lõi dùng gradient: lấy đạo hàm của đầu ra tại vị trí t theo đầu vào tại mọi vị trí $s > t$ và đòi hỏi kết quả bằng 0.

d	âm tiết	BPE (0% OOV)
1	1,71	1,66
6	1,66	2,27
13	1,56	1,94
34	1,26	1,55
88	0,94	1,27
167 (dưới nhiều)		1,03

Bảng 1: Tỷ lệ thông tin tương hỗ giữa tiếng Việt và tiếng Anh theo khoảng cách. Cột âm tiết có nhiều loạn do tỷ lệ token ngoài từ điển chênh lệch (tiếng Anh 10,26% so với tiếng Việt 2,59%); cột BPE khử nhiều loạn này.

Attention đạt 0 tuyệt đối nhờ cơ chế che; Hyena đạt dưới 10^{-15} trên mọi biến thể đã kiểm (lần chạy khi viết báo cáo này cho $3,8 \times 10^{-16}$), tức nhân quả tới sai số máy, vì vòng biến đổi Fourier thuận nghịch để lại nhiều làm tròn.

Quan trọng hơn, chúng tôi bổ sung một *phép kiểm ngược*: cố tình cài lỗi nhân quả kinh điển (quên cắt phần đệm phải của tích chập ngắn) và xác nhận tỷ lệ rò rỉ vọt lên khoảng 2. Chênh lệch hơn 15 bậc độ lớn cho thấy bộ kiểm thử thực sự có khả năng phát hiện lỗi, chứ không phải im lặng vì không đủ nhạy.

5 Kết quả

5.1 Cấu trúc thông tin của tiếng Việt so với tiếng Anh

Bảng 1 trình bày tỷ lệ $I_{vi}(d)/I_{en}(d)$ đo với hai cách token hoá. Cột thứ nhất dùng token hoá theo âm tiết với từ điển 16k; cột thứ hai dùng BPE mức byte, không sinh token ngoài từ điển, với đúng 10.000.000 token cho mỗi ngôn ngữ.

Ba nhận xét.

Thứ nhất, với phép đo BPE, tiếng Việt mang nhiều thông tin tương hỗ cục bộ hơn tiếng Anh ở mọi khoảng cách trong vùng đo được, nhưng ưu thế đó tập trung ở khoảng cách ngắn: tỷ lệ đạt 1,66 tới 2,27 ở $d \leq 13$, rồi giảm còn 1,55 ở $d = 34$, 1,27 ở $d = 88$ và 1,03 ở $d = 167$. Với phép đo âm tiết, ưu thế đảo chiều từ $d \approx 88$ (xem nhận xét thứ hai), nên phát biểu này chỉ vững với phép đo BPE. Hình 1 cho thấy trực tiếp điều đó.

Thứ hai, phép đo bằng âm tiết cho thấy hai đường cắt nhau tại $d \approx 88$, nhưng điểm cắt này *biến mất* khi khử token ngoài từ điển. Nó là hiện vật của việc gộp mọi từ hiếm của tiếng Anh vào một ký hiệu duy nhất, làm mất thông tin của tiếng Anh một cách hệ thống. Chúng

Cấu hình	≤ 4	≤ 34	trung vị
uniform	0	0	169,3
logspace	57	145	22,6
corpus, tiếng Việt	142	227	2,95
corpus, tiếng Anh	141	214	2,85

Bảng 2: Số kênh trên tổng 256 có độ dài hiệu dụng dưới ngưỡng cho trước ($L = 512$). Ngưỡng 34 token là nơi chứa 87,5% thông tin với tiếng Việt và 79,9% với tiếng Anh khi đo bằng âm tiết, 88,7% và 83,5% khi đo bằng BPE.

tôi nêu ra để cảnh báo: nếu chỉ đo bằng âm tiết, nhóm đã kết luận sai.

Thứ ba, tín hiệu chìm xuống dưới mức nhiễu ở $d > 121$ với phép đo âm tiết và $d > 167$ với BPE. Mọi phát biểu về khoảng cách xa hơn các mốc này đều không có căn cứ.

5.2 Phân bố kênh bộ lọc

Bảng 2 so sánh phân bố độ dài hiệu dụng của bốn cấu hình.

5.3 Nhánh đối chứng tiếng Anh bác bỏ tính đặc thù ngôn ngữ

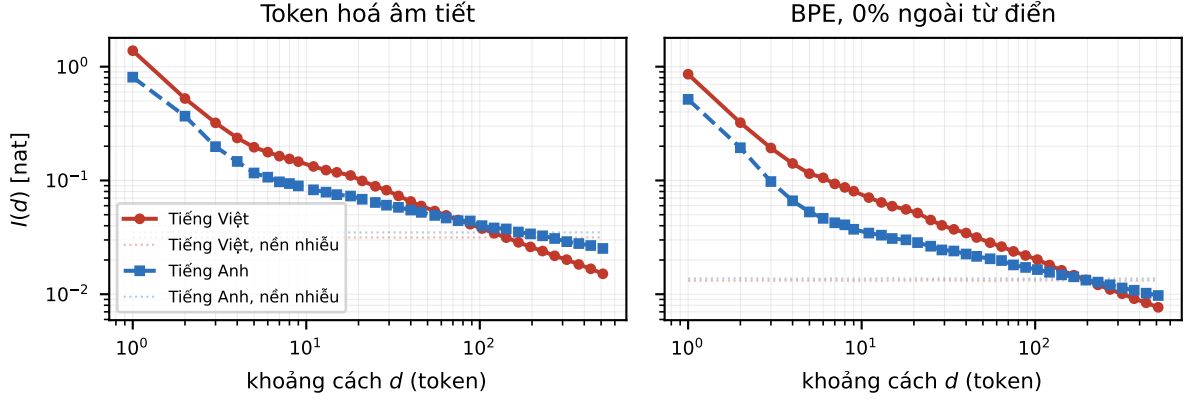
Đây là kết quả quan trọng nhất của báo cáo, và nó đi ngược giả thuyết ban đầu của chúng tôi.

Với nhiều loạn token ngoài từ điển đã được khử, bộ lọc suy ra từ corpus tiếng Việt và từ corpus tiếng Anh gần như trùng nhau: trung vị độ dài hiệu dụng 2,95 so với 2,85 token. Cụ thể, trên 256 kênh đã sắp theo phân vị, độ lệch tương đối trung bình $\text{mean}(|\ell_{vi} - \ell_{en}|/\ell_{en})$ bằng 14,3%; nếu đổi mẫu số sang ℓ_{vi} thì con số là 22,3%, và thước đo đối xứng cho 17,3%. Tương quan Pearson giữa $\log \ell_{vi}$ và $\log \ell_{en}$ là 0,9974, nhưng chỉ nên xem nó là mô tả hình dạng của hai vector phân vị đơn điệu, vì cặp corpus tiếng Việt với đường cơ sở *logspace* vẫn đạt tương quan $\log 0,9377$ dù độ lệch tương đối trung bình đã là 75,8% khi lấy *logspace* làm mẫu số. Các số này được tái sinh bởi `python3 -m hyena_study.analyze`.

Nguyên nhân nằm ở chính công thức ánh xạ: chuẩn hoá $I(d)$ thành phân bố xác suất làm triệt tiêu hệ số tỷ lệ chung, chỉ giữ lại hình dạng, mà hình dạng thì hai ngôn ngữ gần như giống nhau.

5.4 Perplexity trên mô hình ngôn ngữ

Bảng 3 trình bày kết quả chính. Mỗi cấu hình chạy 3 seed với cùng 50 triệu token huấn luyện.



Hình 1: Suy giảm thông tin tương hỗ theo khoảng cách. Đường chấm mảnh là nền nhiễu ước lượng; phần đường cong nằm dưới nó không mang thông tin. Với token hoá âm tiết (trái), hai đường cắt nhau quanh $d \approx 88$; khi khử token ngoài từ điển bằng BPE (phải), điểm cắt biến mất và tiếng Việt nằm trên tiếng Anh trong toàn vùng tin cậy.

Ngôn ngữ	Mô hình	PPL	KTC 95%
Tiếng Việt	Hyena	51,38 $\pm 0,13$	[51,05; 51,71]
	Transformer	63,86 $\pm 0,29$	[63,14; 64,58]
Tiếng Anh	Hyena	55,82 $\pm 0,35$	[54,96; 56,68]
	Transformer	65,43 $\pm 0,18$	[64,99; 65,87]

Bảng 3: Perplexity trên tập kiểm tra, 3 seed cho mỗi cấu hình. Khoảng tin cậy tính theo phân phối t .

Hyena vượt Transformer 19,5% trên tiếng Việt và 14,7% trên tiếng Anh, với khoảng tin cậy tách rời hoàn toàn trong cả hai trường hợp.

Chúng tôi nhấn mạnh rằng kết quả này *mạnh hơn* điều bài báo gốc khẳng định, vốn chỉ tuyên bố Hyena đạt chất lượng *tương đương* Transformer. Cách đọc thận trọng là ở quy mô nhỏ và ngân sách ngắn này, thiên lệch quy nạp của tích chập tỏ ra có lợi. Quan trọng hơn, cả hai kiến trúc dùng chung learning rate và lịch huấn luyện chứ không tinh chỉnh riêng, nên phát biểu đúng là “Hyena vượt Transformer ở cùng siêu tham số và cùng quy mô này”.

Đáng chú ý là khoảng cách trên tiếng Việt lớn hơn trên tiếng Anh, phù hợp với kết quả đo thông tin tương hỗ ở mục trước.

5.5 Ảnh hưởng của đơn vị token hoá

Ở cùng lượng ký tự, tiếng Việt sinh nhiều token hơn: trên chính corpus huấn luyện, 4,118 ký tự trên mỗi token so với 5,100 của tiếng Anh, tỉ lệ 1,239.

Chỉ xét tiếng Việt, với token hoá âm tiết Hyena đạt 51,38 so với 63,86 của Transformer,

Cấu hình	PPL	Δ	KTC 95%	Tách
Gốc (bậc 2, đủ)	51,38		[51,05; 51,71]	
Bỏ cửa sổ suy giảm	53,30	+1,92	[52,24; 54,35]	có
Bậc $N = 1$	53,11	+1,73	[51,72; 54,49]	có*
Bỏ kích hoạt sine	51,80	+0,42	[51,49; 52,10]	không
Bậc $N = 3$	51,63	+0,25	[49,51; 53,76]	không
Bỏ positional emb	50,79	-0,59	[50,14; 51,45]	không

Bảng 4: Ablation trên tiếng Việt, 2 seed cho mỗi nhánh (cấu hình gốc: 3 seed). Khoảng tin cậy 95% theo phân phối t ($n = 2$: $t = 12,706$; $n = 3$: $t = 4,303$); chỉ những dòng có khoảng tách rời khỏi cấu hình gốc mới được rút ra kết luận. *Tách rời nhưng biên rất sát (0,016 PPL).

tỉ lệ 1,243; với BPE là 80,02 so với 96,22, tỉ lệ 1,202 (3 seed mỗi ô). Ở đây có một cạm bẫy diễn giải: perplexity *không so sánh được giữa hai bộ token hoá khác nhau* vì từ điển khác nhau cho entropy trên mỗi token khác nhau, nên 51,38 với âm tiết và 80,02 với BPE *không* có nghĩa là âm tiết tốt hơn. Đại lượng so sánh được là tỉ lệ giữa hai mô hình trong cùng một bộ token hoá. Lợi thế tương đối của Hyena lớn hơn khi token hoá theo âm tiết (1,243) so với BPE (1,202), đúng chiều với giả thuyết rằng chuỗi dài hơn làm lợi thế dưới bậc hai có giá trị hơn. Tuy nhiên chênh lệch giữa hai tỉ lệ nhỏ và chúng tôi không ước lượng khoảng tin cậy cho tỉ lệ, nên đây chỉ là dấu hiệu, chưa phải bằng chứng.

Ngôn ngữ	Khởi tạo	PPL	KTC 95%
Tiếng Việt	uniform	51,38	[51,05; 51,71]
	logspace	50,48	[50,33; 50,62]
	corpus	50,19	[49,93; 50,45]
Tiếng Anh	uniform	55,82	[54,96; 56,68]
	logspace	54,97	[54,56; 55,37]
	corpus	54,91	[54,53; 55,29]

Bảng 5: Ảnh hưởng của cách khởi tạo phân bố suy giảm, 3 seed cho mỗi ô.

5.6 Ablation các thành phần của Hyena

Chỉ hai nhánh cho khác biệt có ý nghĩa thống kê.

Bỏ cửa sổ suy giảm gây thiệt hại lớn nhất, xác nhận vai trò của số hạng Window(t) trong công thức (2). Hạ bậc xuống $N = 1$ cũng gây thiệt hại, phù hợp với nhận xét trong bài báo gốc rằng Hyena bậc 1 tương ứng với GSS, vốn yếu hơn H3 trên các tác vụ nhỏ. Tuy nhiên kết luận này mong manh hơn nhiều: khoảng tin cậy của $N = 1$ chỉ tách khỏi cấu hình gốc 0,016 PPL, nên thêm một seed hoàn toàn có thể làm hai khoảng chồng lấn trở lại. Ngoài ra, hạ bậc xuống $N = 1$ cũng làm mất 331.776 tham số (4,4% mô hình), nên thiệt hại +1,73 PPL trộn lẫn hai nguyên nhân — mất một tầng cổng đệ quy và mất tham số — mà thiết kế hiện tại không tách riêng được.

Ba nhánh còn lại có khoảng tin cậy chồng lấn với cấu hình gốc nên chúng tôi *không* kết luận. Đáng chú ý là bỏ positional embedding cho perplexity thấp hơn cấu hình gốc, phù hợp với lập luận rằng bộ lọc tích chập đã mang sẵn thông tin vị trí, nhưng với 2 seed chúng tôi không đủ căn cứ khẳng định đây là cải thiện thật.

5.7 Khởi tạo bộ lọc từ corpus

Kết quả trong Bảng 5 tách thành hai phần rõ rệt.

Trước hết cần nói rõ tiêu chí. Chúng tôi coi một khác biệt là kết luận được khi hai khoảng tin cậy 95% không chồng lấn. Đây là tiêu chí bảo thủ hơn kiểm định hai mẫu ở mức 0,05; với số phép so sánh trong mục này, nó xấp xỉ một mức đã hiệu chỉnh cho so sánh bội.

Phần thứ nhất là khẳng định được: cả logspace lẫn corpus đều vượt uniform trên cả hai ngôn ngữ. Với tiếng Việt khoảng cách là 0,90 PPL và hai khoảng tin cậy tách rời. Với tiếng

Anh khoảng cách là 0,85 PPL cho logspace và 0,90 PPL cho corpus, cùng chiều và cùng độ lớn, nhưng hai khoảng tin cậy chồng lấn vì ba seed của uniform phân tán rộng hơn; kiểm định Welch hai mẫu cho $p \approx 0,03$, tức vẫn có ý nghĩa nếu xét một mình ở mức 0,05 nhưng không qua được tiêu chí bảo thủ nói trên. Nói cách khác, khoảng giá trị của α thực sự quan trọng, và một khoảng chọn tùy tiện gây thiệt hại đo được.

Phần thứ hai là khẳng định *không* được: corpus chỉ hơn logspace 0,285 PPL trên tiếng Việt và 0,05 PPL trên tiếng Anh, và khoảng tin cậy chồng lấn trong cả hai trường hợp. Chúng tôi không có căn cứ kết luận rằng suy phân bố suy giảm từ thống kê corpus tốt hơn một lựa chọn cách đều theo thang log. Chúng tôi nêu rõ một điểm có thể bị chất vấn: riêng trên tiếng Việt, kiểm định Welch cho $p = 0,024$, tức sẽ có ý nghĩa nếu xét một mình ở mức 0,05. Nhưng mục này thực hiện sáu phép so sánh cặp, và sau hiệu chỉnh Bonferroni ($\alpha = 0,0083$) con số đó không còn qua ngưỡng, trong khi hai phép so sánh với uniform trên tiếng Việt thì qua ($p = 0,0024$ và $p = 0,0003$). Vì vậy chúng tôi giữ kết luận phủ định.

Thêm nữa, phần cải thiện nhỏ ấy xuất hiện trên *cả hai* ngôn ngữ, cùng chiều tuy không cùng độ lớn. Đây chính là điều mà phân tích ở mục trước đã dự đoán từ chỗ hai bộ lọc gần như trùng nhau, và nó khép lại lập luận: nếu có lợi ích thì lợi ích đó không mang tính đặc thù ngôn ngữ.

5.8 Hiệu năng theo độ dài chuỗi

Ở $L = 512$, trên chính các lần chạy tiếng Việt của Bảng 3 (Tesla T4), chúng tôi đo được Transformer đạt 138,5 nghìn token mỗi giây còn Hyena đạt 92,0 nghìn, tức Transformer *nhANH hơn* 1,51 lần. Nhánh tiếng Anh chạy trên một máy chủ khác cho cùng chiều với biên độ nhỏ hơn: 122,4 nghìn so với 85,7 nghìn, tức 1,43 lần. Kết quả này không mâu thuẫn với bài báo gốc: các con số tăng tốc được công bố ở đó áp dụng cho $L \geq 8K$, nơi chi phí bậc hai của attention mới lấn át chi phí cố định của biến đổi Fourier. Chúng tôi nêu rõ điều kiện này để tránh trích dẫn sai.

Khi đo hiệu năng, việc chọn đường cơ sở attention nào cũng quyết định kết luận. Bài báo gốc so với FlashAttention (Dao et al., 2022) và tự ghi rõ rằng cài đặt này đã nhanh hơn

L	tiền		tiền và lùi	
	sdpa	ngây thơ	sdpa	ngây thơ
256	0,16	0,55	0,23	0,47
512	0,22	0,92	0,27	0,82
1024	0,28	1,56	0,33	1,60
2048	0,37	3,77	0,56	3,13
4096	0,69	8,30	1,02	6,47
8192	1,19	OOM	1,88	OOM

Bảng 6: Mức tăng tốc của Hyena so với hai đường cơ sở attention, tính bằng thương số thời gian chạy (số lớn hơn 1 nghĩa là Hyena nhanh hơn). Cột **sdpa** là kernel tiết kiệm bộ nhớ của PyTorch, cột “ngây thơ” là cài đặt materialise ma trận $L \times L$. Cài đặt ngây thơ hết bộ nhớ ở $L = 8192$.

attention thông thường trong PyTorch từ 2 tới 4 lần. Vì vậy chúng tôi báo cáo cả hai đường cơ sở: một cài đặt materialise ma trận $L \times L$ và một cài đặt dùng kernel tiết kiệm bộ nhớ của PyTorch. Chỉ so với đường cơ sở thứ nhất sẽ thổi phồng mức tăng tốc nhiều lần.

Bảng 6 cho thấy điểm hoà vốn phụ thuộc mạnh vào đường cơ sở. So với cài đặt ngây thơ, Hyena vượt lên từ khoảng $L = 1024$. So với kernel tiết kiệm bộ nhớ, Hyena chỉ vượt lên ở $L = 4096$ với lượt tiến và lùi, và ở $L = 8192$ với lượt tiến.

Con số đáng chú ý nhất là 1,88 lần ở $L = 8192$ cho lượt tiến và lùi. Bài báo gốc công bố mức tăng tốc 2 lần so với FlashAttention ở độ dài 8K. Kết quả của chúng tôi tiệm cận con số đó, dù dùng `torch.fft` thuần thay cho CUDA kernel tùy chỉnh. Đây là phần tái hiện thành công nhất của nghiên cứu này.

Về bộ nhớ, cài đặt ngây thơ tiêu thụ 30,6, 83,1, 283,6, 1074,5 và 4201,6 MB khi L tăng gấp đôi từ 256 tới 4096, tức xấp xỉ gấp bốn mỗi lần, đúng dạng bậc hai, và hết bộ nhớ ở 8192. Hyena tiêu thụ 36,5, 66,1, 117,7, 224,7, 438,7 và 865,8 MB, tức xấp xỉ gấp đôi mỗi lần, đúng dạng tuyến tính.

5.9 Chẩn đoán trí nhớ tầm xa

Perplexity không phát hiện được việc mô hình mất khả năng tầm xa: một mô hình chỉ nhìn vài token vẫn đạt perplexity khá tốt trên văn bản tự nhiên. Vì bộ lọc suy từ corpus có trung vị độ dài hiệu dụng chỉ 2,9 token, chúng tôi bổ sung tác vụ associative recall, vốn là công cụ chẩn đoán của chính bài báo gốc.

Trước hết chúng tôi xác nhận tác vụ giải được: attention hai lớp đạt độ chính xác 1,000

ở độ dài 65 và giảm còn 0,196 ở độ dài 129, với mức đoán mò 0,100; và lịch warmup là yếu tố quyết định, vì cùng 18.720 bước và cùng learning rate, có warmup đạt 1,000 còn không có warmup chỉ đạt 0,187. Ở độ dài 65, mọi cấu hình Hyena đều kém hơn attention và dao động rất mạnh giữa các seed: 0,564 với uniform và 0,170 với logspace (2 seed mỗi cấu hình). Độ lệch giữa các seed lớn hơn khoảng cách giữa bất kỳ cặp cấu hình nào, nên chúng tôi *không* xếp hạng được các biến thể bộ lọc trên tác vụ này.

Với cấu hình suy từ corpus, chúng tôi gộp 5 seed từ hai ngân sách huấn luyện (2 seed ở 18.720 bước và 3 seed ở 37.440 bước, cùng độ dài 65 và cùng cấu hình tác vụ); việc gộp này là một lựa chọn phương pháp nên chúng tôi nêu rõ. Phân bố thu được *lượng cực*: ba lần đạt 0,922; 0,981; 0,987 và hai lần sập về 0,092 và 0,141, không có giá trị trung gian. Gấp đôi ngân sách làm các lần thành công tốt lên (0,922 lên 0,98) nhưng *không* khử được kiểu hổng (độ lệch 0,415 xuống 0,397), tức thất bại tối ưu hoá chứ không phải hội tụ chậm: Hyena có đủ năng lực biểu diễn ở độ dài 65. Với phân bố lượng cực, báo cáo trung bình kèm độ lệch chuẩn là sai lệch; đại lượng đúng là tỉ lệ thành công 3 trên 5 và độ chính xác khi thành công 0,963.

6 Thảo luận

Phát hiện có giá trị nhất của nghiên cứu này là một kết quả phủ định do chính thiết kế đối chứng của chúng tôi tạo ra. Giả thuyết ban đầu cho rằng bộ lọc suy từ thống kê corpus tiếng Việt sẽ mang tính đặc thù ngôn ngữ. Nhánh đối chứng tiếng Anh bác bỏ giả thuyết đó bằng số liệu: hai bộ lọc lệch nhau 14,3% trong khi cùng lệch khoảng 76% so với đường cơ sở.

Nếu không có nhánh đối chứng, nhóm đã có thể công bố một khẳng định về đặc thù tiếng Việt mà chỉ cần áp dụng cùng quy trình lên tiếng Anh là bác bỏ được.

Kết quả huấn luyện xác nhận đúng dự đoán rút ra từ phân tích bộ lọc trước khi chạy thí nghiệm: chuyển từ uniform sang logspace thu được khoảng 0,9 PPL, còn chuyển tiếp sang corpus chỉ thêm 0,05 tới 0,29 PPL với khoảng tin cậy chồng lấn.

Việc bổ sung đường cơ sở logspace sau khi phát hiện uniform đặt toàn bộ kênh ra ngoài vùng có thông tin là quyết định tới kết luận:

nếu chỉ so với `uniform`, nhóm đã quy cả 1,19 PPL cho đóng góp đề xuất trong khi phần lớn chỉ là hệ quả của việc sửa một đường cơ sở đặt kém.

Phần đặc thù tiếng Việt còn đứng vững nằm ở chỗ khác: ở khoảng cách ngắn tiếng Việt mang lượng thông tin tương hỗ cục bộ gấp 1,7 tới 2,3 lần tiếng Anh, và trực âm tiết so với BPE là phân biệt chỉ tồn tại ở dạng này với chữ Quốc ngữ.

Hiện tượng lưỡng cực ở tác vụ associative recall gợi ra một hướng đáng theo đuổi: vấn đề của Hyena ở quy mô nhỏ có thể nằm ở độ ổn định tối ưu hoá chứ không phải ở năng lực biểu diễn, vì hai trong ba lần chạy đạt tới 0,98.

Hạn chế

Quy mô mô hình của chúng tôi (khoảng 7,5 triệu tham số) nhỏ hơn ba bậc so với quy mô 355 triệu tham số trong bài báo gốc. Mọi kết luận đều gắn với quy mô này và không được ngoại suy.

Chúng tôi dùng `torch.fft` thuần thay cho CUDA kernel tùy chỉnh của nhóm tác giả, nên các con số tốc độ của chúng tôi không so trực tiếp được với con số trong bài báo gốc.

Bản tái hiện của chúng tôi *chưa đạt được* kết quả associative recall của bài báo gốc, vốn báo cáo 97,2% ở độ dài 131.072. Đây là hạn chế của bản tái hiện ở ngân sách hiện có, không phải bằng chứng phản bác bài báo gốc.

Do phân bố lưỡng cực và số seed hạn chế, chúng tôi *không* xếp hạng được các biến thể bộ lọc trên tác vụ associative recall. Phân biệt được tỉ lệ thành công giữa các cấu hình đòi hỏi khoảng 10 seed cho mỗi cấu hình, vượt ngân sách tính toán của đồ án.

Corpus được lấy mẫu ở chế độ streaming, trong đó việc xáo trộn chỉ diễn ra trong một bộ đệm hữu hạn chứ không xáo toàn cục, nên mẫu lệch về phía đầu tập dữ liệu. Nặng hơn, phép lấy mẫu này không tái lập được giữa các phiên bản thư viện `datasets`. Vì vậy nhánh tiếng Anh phải được lấy mẫu lại (110.000 tài liệu, `datasets` phiên bản 4.0.0, ghi trong mỗi tệp kết quả) rồi cắt xuống đúng ngân sách token của nhánh tiếng Việt, còn nhánh tiếng Việt giữ mẫu gốc; đổi lại hai ngôn ngữ không còn chung một lần lấy mẫu.

Một hạn chế riêng của cách sinh α nằm ở phép ánh xạ từ $I(d)$ sang phân bố độ dài hiệu

dụng. Hàm `alphas_from_mi` coi mỗi điểm trên lưới log của d là một khối lượng xác suất rồi chuẩn hoá trực tiếp, chưa nhân với bề rộng ô quanh từng điểm; vì lưới log dày ở khoảng cách ngắn, quy ước này kéo phân bố về phía các lag nhỏ. Đo lại trên cùng dữ liệu nhưng có nhân bề rộng ô, trung vị độ dài hiệu dụng của tiếng Việt chuyển từ 2,95 lên 90,54 token (số kênh ≤ 4 giảm từ 142 xuống 39, số kênh ≤ 34 từ 227 xuống 87), tiếng Anh từ 2,85 lên 160,45 token (141 xuống 27, và 214 xuống 57). Đây chỉ là kiểm tra độ nhạy của cách diễn giải, không phải kết quả huấn luyện mới; vì vậy phát biểu “bộ lọc corpus có trung vị khoảng 3 token” phải hiểu theo đúng quy ước rồi rạc đang dùng trong code.

Cuối cùng, so sánh “cùng số ký tự” giữa hai ngôn ngữ không phải là “cùng lượng nội dung”. Muốn khẳng định chuỗi tiếng Việt dài hơn ở cùng nội dung cần văn bản song ngữ song song, thứ chúng tôi không có.

7 Kết luận

Chúng tôi cài đặt lại toán tử Hyena kèm bằng chứng kiểm thử về tính đúng đắn và áp dụng cho mô hình ngôn ngữ tiếng Việt. Ở cùng ngân sách tham số và cùng siêu tham số, Hyena đạt perplexity thấp hơn Transformer 19,5% trên tiếng Việt và 14,7% trên tiếng Anh, và vượt kernel attention tiết kiệm bộ nhớ 1,88 lần ở độ dài 8192 cho lượt tiến và lùi. Ablation cho thấy cửa sổ suy giảm và bậc đệ quy là hai thành phần có ảnh hưởng đo được, còn kích hoạt sine và positional embedding thì không phân biệt được ở số seed hiện có.

Đóng góp đề xuất cho kết quả hai mặt. Khoảng giá trị của tham số suy giảm thực sự quan trọng: chuyển từ một khoảng đặt tùy tiện sang một lựa chọn có nguyên tắc thu được khoảng 0,9 PPL. Nhưng việc suy khoảng đó từ thống kê corpus không mang lại lợi ích đo được so với một lựa chọn cách đều theo thang log, và phần chênh lệch nhỏ còn lại tuy cùng chiều trên cả hai ngôn ngữ nhưng không cùng độ lớn (0,285 PPL với tiếng Việt, 0,05 PPL với tiếng Anh). Chúng tôi báo cáo kết quả phủ định này cùng toàn bộ giới hạn của bản tái hiện, vì chính nhánh đối chứng do chúng tôi thiết kế đã dự đoán và sau đó xác nhận nó.

References

- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Daniel Y. Fu, Tri Dao, Khaled K. Saab, Armin W. Thomas, Atri Rudra, and Christopher Ré. 2023. Hungry hungry hippos: Towards language modeling with state space models. In *International Conference on Learning Representations (ICLR)*.
- Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. 2020. The pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*.
- Albert Gu and Tri Dao. 2023. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*.
- Albert Gu, Karan Goel, and Christopher Ré. 2022. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations (ICLR)*.
- Harsh Mehta, Ankit Gupta, Ashok Cutkosky, and Behnam Neyshabur. 2022. Long range language modeling via gated state spaces. *arXiv preprint arXiv:2206.13947*.
- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. Pointer sentinel mixture models. In *International Conference on Learning Representations (ICLR)*.
- Dat Quoc Nguyen and Anh Tuan Nguyen. 2020. PhoBERT: Pre-trained language models for Vietnamese. In *Findings of the Association for Computational Linguistics: EMNLP 2020*.
- Michael Poli, Stefano Massaroli, Eric Nguyen, Daniel Y. Fu, Tri Dao, Stephen Baccus, Yoshua Bengio, Stefano Ermon, and Christopher Ré. 2023. Hyena hierarchy: Towards larger convolutional language models. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research*.
- David W. Romero, Anna Kuzina, Erik J. Bekkers, Jakub M. Tomczak, and Mark Hoogendoorn. 2022. Ckconv: Continuous kernel convolution for sequential data. In *International Conference on Learning Representations (ICLR)*.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*.
- Vincent Sitzmann, Julien N. P. Martel, Alexander W. Bergman, David B. Lindell, and Gordon Wetzstein. 2020. Implicit neural representations with periodic activation functions. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*.